Fleet Manager

RELATÓRIO TÉCNICO · ETAPA DE TESTES

Relatório de Testes do Fleet Manager

Estratégia, execução e resultados dos testes do sistema de gestão de frotas: código, segurança, integração contínua, observabilidade, carga, volume, resiliência e produção.

373	86,4%	207 mil	0
testes automatizados, todos passando	das linhas da API executadas pelos testes	requisições de carga, nenhuma com erro	vulnerabilidades nas dependências

Data **20 de outubro de 2026**

Ambiente de teste **Supabase local (Docker)**

Sistema **Fleet Manager** · web, multiempresa

Produção **Vercel (São Paulo)** · **Supabase**

01 Estratégia e resultado geral
02 Ambiente e como ler as métricas
03 Análise estática e build
04 Testes unitários e de integração
05 Testes de ponta a ponta
06 Segurança
07 Cobertura de código
08 Integração contínua (CI/CD)
09 Observabilidade
10 Testes de carga
11 Teste de volume
12 Resiliência
13 Telas no navegador
14 Verificação em produção
15 Exemplos de código
16 Perguntas frequentes

01 · ESTRATÉGIA E RESULTADO GERAL

Sete camadas de verificação, do código à produção

A validação foi organizada em camadas, da mais barata e rápida à mais próxima do uso real. Cada camada responde a uma pergunta diferente sobre o sistema, e todas passaram.

CAMADA	PERGUNTA QUE RESPONDE	COMO	RESULTADO
Análise estática	O código está correto antes de rodar?	TypeScript, ESLint, build, npm audit	0 erros
Unitários e integração	Cada regra de negócio funciona?	Vitest · 246 testes	246 / 246
Ponta a ponta (E2E)	As peças funcionam montadas, por HTTP?	Vitest + Supertest · 127 casos	127 / 127
Integração contínua	Tudo isso roda sozinho a cada mudança?	GitHub Actions · 9 etapas	~2 min
Carga e volume	Aguenta muitos acessos e muitos dados?	autocannon · até 200 conexões · 50 mil despesas	0 erros
Resiliência	Como reage se o banco cair?	Banco pausado com a API no ar	503 e recupera
Telas	A interface abre para cada perfil?	Chrome automatizado · 5 perfis	30 aberturas
Produção	O sistema publicado responde como o testado?	Rotas, permissões, telas e carga na Vercel	0 erros

373
testes automatizados (246 + 127)

195.611
requisições de carga no ambiente de teste, 0 erros

11.840
requisições de carga em produção, 0 erros

55
aberturas de tela no navegador (teste e produção), 0 erros

02 · AMBIENTE E MÉTRICAS

Onde os testes rodam e como ler os números

Ambiente de teste

- **Supabase local no Docker**: PostgreSQL 17, autenticação e armazenamento, isolados da produção.
- **Chave de assinatura ES256 própria**, porque a API só aceita tokens assimétricos verificados pela chave pública.
- **Migrations e seed** criam o banco e duas empresas de demonstração com todos os perfis.
- A suíte de ponta a ponta **recusa rodar fora do ambiente local**, para nunca gravar em produção.

Ferramentas

- **Vitest 4** e **Supertest**: testes unitários, de integração e de ponta a ponta.
- **autocannon**: carga, com tokens reais de cada perfil.
- **Chrome automatizado** (Playwright): telas e tempos de abertura.
- **GitHub Actions**: integração contínua.
- **Vercel**: publicação contínua, erros agrupados e firewall.

Como ler as métricas deste relatório

MÉTRICA	O QUE SIGNIFICA
Req/s (vazão)	Quantas requisições a API atendeu por segundo durante o teste. Mede capacidade.
Conexões simultâneas	Quantos clientes pedem ao mesmo tempo, sem pausa entre um pedido e outro. 50 conexões contínuas equivalem a muito mais que 50 pessoas usando o sistema, porque uma pessoa real lê a tela entre um clique e outro.
Mediana (p50)	Metade das respostas chegou nesse tempo ou menos. É o tempo típico que o usuário sente.
p90 e p99	90% e 99% das respostas chegaram nesse tempo ou menos. Mostram a regularidade: quanto mais perto da mediana, mais estável.
Erros	Requisições que falharam ou receberam status fora de 2xx sem dever. Em todos os testes de carga deste relatório: zero.
Cobertura de linhas	Percentual das linhas da API executadas ao menos uma vez pelos testes. **Funções**: funções chamadas. **Ramos**: caminhos de if/else percorridos.
Status HTTP	`200` sucesso · `401` sem identificação válida · `403` sem permissão · `404` inexistente ou fora do alcance · `503` serviço temporariamente indisponível.

03 · ANÁLISE ESTÁTICA E BUILD

O código é verificado antes de rodar

Análise estática verifica o código sem executá-lo e encontra erros antes de o sistema subir. É a primeira etapa do CI.

VERIFICAÇÃO	COMANDO	O QUE CONFERE	RESULTADO
Tipagem da API	`tsc --noEmit`	Cada valor tem o tipo esperado, do controller ao banco	0 erros
Tipagem do frontend	`tsc --noEmit`	Telas, hooks e chamadas à API com os tipos certos	0 erros
Lint	`npm run lint`	Padrões do projeto em .ts, .tsx e .mts	0 avisos
Build de produção	`npm run build`	O frontend gera a versão otimizada que vai ao ar	0,9 s
Dependências	`npm audit`	Vulnerabilidades conhecidas nas bibliotecas usadas	0

Stack verificada

React 18, Vite 8, React Router 7 e Tailwind CSS 4 no frontend; Node.js, Express 4, postgres.js e Zod na API; Vitest 4 e typescript-eslint 8 nas ferramentas. Todas as bibliotecas estão nas versões atuais, sem vulnerabilidade conhecida.

04 · TESTES UNITÁRIOS E DE INTEGRAÇÃO

246 testes das regras de negócio

Teste unitário · 206 testes

Verifica uma regra isolada, com o banco substituído por uma versão simulada. É rápido e aponta exatamente onde algo mudou.

Teste de integração · 40 testes

Executa as consultas reais num PostgreSQL com as migrations aplicadas: restrições, isolamento por empresa e regras que dependem do banco.

GRUPO	O QUE VERIFICA	ARQUIVOS
Serviços	Veículos, motoristas, vínculos com período, despesas, manutenções, documentos, usuários, cadastro e painel	9
Middlewares	Autenticação, permissão por papel, validação de entrada, tratamento de erros, tempo-limite e registro de requisições	5
Bibliotecas e rotinas	Anexos, verificação de saúde do banco, rotina diária de alertas, controller de usuários	4
Integração com o banco	Consultas reais com isolamento por empresa, unicidade e restrições do esquema	1

246 / 246
passando, em cerca de 2 segundos

Sem .env
a suíte roda em qualquer máquina: os testes de banco são pulados, não quebram

Ordem livre
continua passando depois da suíte E2E, no mesmo banco

05 · TESTES DE PONTA A PONTA

127 casos pelo caminho real da requisição

O teste de ponta a ponta (E2E) envia requisições HTTP reais ao servidor e confere a resposta final, passando por todas as camadas: rota, autenticação, permissão, controller, serviço, repositório e banco. Os tokens são emitidos pelo próprio Supabase Auth; o login não é simulado. A suíte grava de verdade no banco local e pode ser repetida sem limpar o banco.

ÁREA	O QUE VERIFICA	CASOS
INF · Infraestrutura	Saúde da API e do banco, CORS, preflight, rotina de alertas protegida por segredo	5
AUT · Autenticação	Token ausente, malformado, forjado, com algoritmo trocado ou sem assinatura; perfis pendente e recusado	13
RBAC · Permissões	Matriz de 20 ações negadas e 7 permitidas por papel	27
ISO · Isolamento	Dados de outra empresa respondem 404; o painel soma só a própria frota	8
OPE · Motorista	Vê só os próprios veículos, ficha e lançamentos	6
VEI · Veículos	Validação, placa única, vínculo de motorista, inativar antes de excluir	7
DES · Despesas	Valor, categoria, precisão monetária, cancelamento com motivo, reativação, histórico	11
MAN · Manutenções	Agendar, concluir, cancelar e reativar	4
DOC · Documentos	Anexo por URL assinada de 60 s, extensões aceitas, armazenamento fechado	11
CAD · Cadastro	Empresa nova, cadastro real, CPF repetido, aprovação, empresa inativa	14
VAL · Validação	Ordenação fechada, injeção SQL, identificador inválido, JSON malformado	11
PAG · Paginação	Tamanho de página, total no cabeçalho, páginas sem repetição, filtros, limites	10

Cada caso tem um identificador (por exemplo, `AUT-04`) que aparece no nome do teste e na saída do Vitest, para rastrear exatamente o cenário.

06 · SEGURANÇA

Os ataques conhecidos são testados e recusados

Autenticação · 13 casos

- Token ES256 assinado com outra chave e o mesmo identificador de chave
- Token HS256 com o segredo antigo (confusão de algoritmo)
- Token com `alg: none`, sem assinatura
- Chave pública usada como se fosse token de usuário
- Perfis pendente e recusado barrados

Permissões · 27 casos

- Matriz com motorista, gerente, administrador e super administrador
- 20 ações que cada papel não pode fazer recebem 403
- 7 ações permitidas recebem 200
- O cabeçalho de empresa é ignorado para perfil comum

Isolamento entre empresas · 8 casos

- Recurso de outra empresa responde 404, sem revelar que existe
- Não é possível lançar despesa em veículo de outra empresa
- Autor e empresa enviados no corpo são ignorados: quem define é o servidor

Entradas hostis · 11 casos

- Tentativa de injeção SQL nos filtros e na ordenação
- Identificador inválido na URL responde 404
- JSON malformado responde 400, corpo grande demais 413

Duas camadas em produção

Em produção, o token sem assinatura nem chega à aplicação: o firewall da Vercel o barra antes (`403` , `X-Vercel-Mitigated: deny`). Tokens inválidos que passam pelo firewall são recusados pela API com `401` . Os anexos ficam num armazenamento privado e só saem por link assinado válido por 60 segundos.

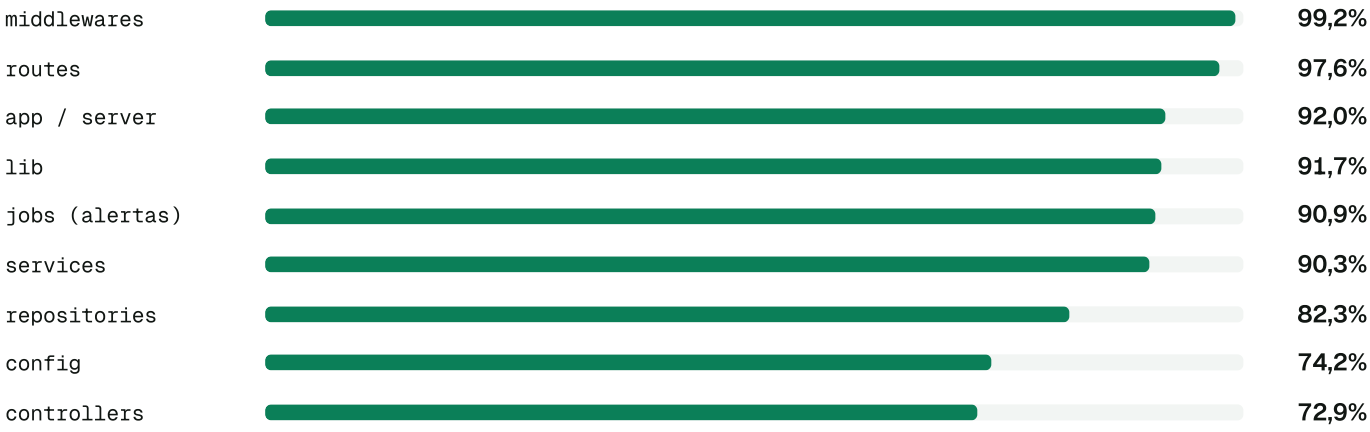
07 · COBERTURA DE CÓDIGO

86,4% das linhas da API executadas pelos testes

Cobertura mede quanto do código os testes executam. Foi medida com o Vitest (provedor V8) sobre todo o código-fonte da API, sem os próprios arquivos de teste, com as duas suítes juntas.

86,4%	87,3%	84,0%	74,1%
linhas	funções	instruções	ramos de if/else

Linhas cobertas por camada



As camadas que concentram as regras de segurança e de negócio, middlewares, rotas e serviços, estão acima de 90%. A suíte de ponta a ponta é a que cobre a camada HTTP: rotas, controllers e o servidor.

Cada mudança é testada sozinha antes de ir ao ar

CI (integração contínua) roda todas as verificações automaticamente a cada pull request e a cada push na `main`. **CD (entrega contínua)** publica a versão aprovada: a Vercel faz a publicação a cada push, com prévia para cada branch e retorno a qualquer versão anterior com um clique. Nas 16 publicações registradas, todas concluíram.

01 **Dependências** `npm ci` · 36 s	02 **Tipagem** API e frontend · 11 s	03 **Lint** .ts, .tsx, .mts · 4 s
04 **Build do frontend** versão de produção · 7 s	05 **Supabase local** criado do zero no runner · 47 s	06 **Migrations e seed** 6 migrations · 5 s
07 **Suíte padrão** 246 testes · 4 s	08 **Suíte de ponta a ponta** 127 casos · 7 s	09 **Auditoria** `npm audit` · 2 s

O pipeline completo leva cerca de **2 minutos** e não usa nenhuma credencial de produção: o banco é criado do zero dentro do próprio job e descartado ao final. Todas as etapas foram executadas do zero, na mesma ordem e com os mesmos comandos do arquivo, e passaram.

.github/workflows/testes.yml (trecho)

```yaml
on:
  pull_request:
  push:
    branches: [main]
jobs:
  validar:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npx tsc --noEmit -p apps/api && npx tsc --noEmit -p apps/web
      - run: npm run lint
      - run: npm run build --workspace=apps/web
      - uses: supabase/setup-cli@v1
      - name: Supabase local
        run: |
          echo '[]' > supabase/signing_keys.json
          supabase gen signing-key --algorithm ES256 --append
          supabase start -x studio,imgproxy,vector,logflare,edge-runtime,realtime,mailpit,supavisor,postgres-meta
      - run: npm run db:migrate && npm run db:seed
      - run: npm run test:api          # 246 testes
      - run: npm run test:e2e          # 127 casos
      - run: npm audit --audit-level=high
```

09 · OBSERVABILIDADE

O sistema diz como está, a cada requisição

Observabilidade é a capacidade de saber, de fora, se o sistema está no ar, onde falha e quão rápido responde. Cada sinal abaixo foi testado com o banco disponível e com o banco fora do ar.

SINAL	COMO FUNCIONA	RESULTADO DO TESTE
Verificação de saúde `GET /health`	Consulta o banco com prazo de 3 s e informa o estado de cada parte	Banco disponível: `200`, `"database": "ok"`, em 3 ms. Banco fora: `503`, `"database": "unavailable"`, em 3 s.
Registro por requisição	Uma linha por requisição atendida: método, caminho, status e duração	Todas as requisições dos testes aparecem no log, inclusive as de erro.
Tempo-limite	Requisição que passa de 10 s recebe `503` e fica registrada	Com o banco fora, as rotas respondem 503 em 10 s; nada fica preso.
Erros agrupados	A Vercel agrupa os erros da aplicação por tipo, com rota e horário	Nenhum erro registrado em produção durante os testes.
Auditoria de negócio	Cada alteração grava autor, data, campos alterados e motivo	Verificada pelo teste DES-10; o histórico aparece na própria tela.
Firewall	Proteção da Vercel antes da aplicação	Token sem assinatura barrado com `X-Vercel-Mitigated: deny`.

Log da API durante o teste de resiliência: o banco sai do ar e volta

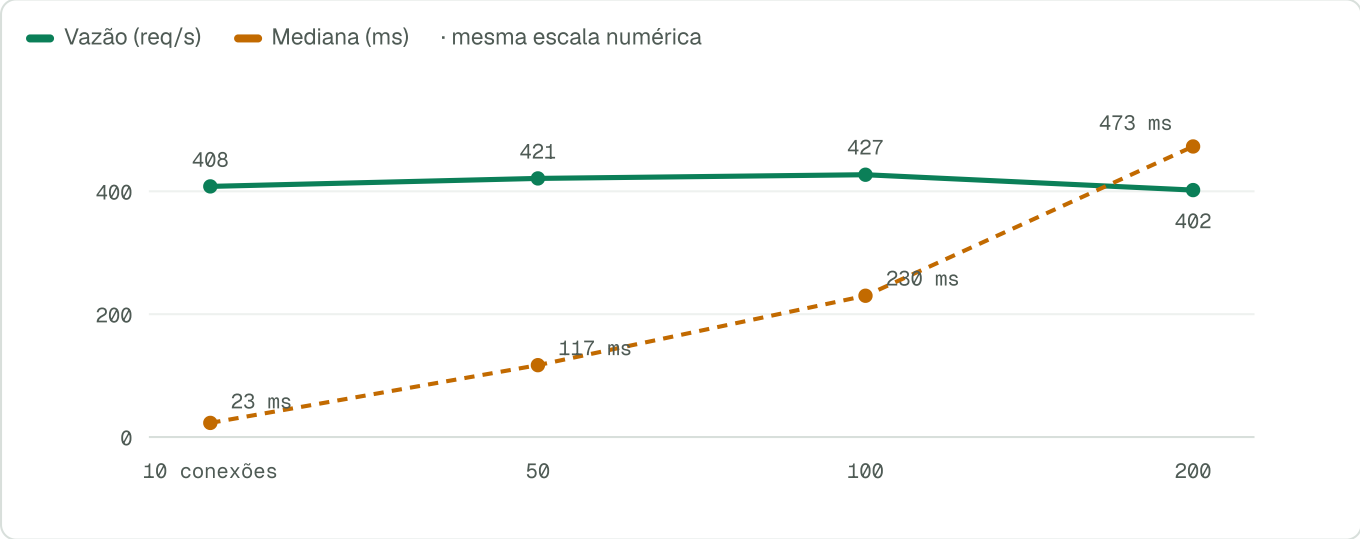
```
[req] GET /health 200 2ms
[req] GET /health 503 3016ms                    # banco fora: a saúde acusa em 3 s
[timeout] GET /api/vehicles passou de 10000ms
[req] GET /api/vehicles 503 10015ms             # a rota não fica presa
[req] GET /api/vehicles 401 1ms                 # o processo segue atendendo
[req] GET /health 200 2ms                       # banco de volta
[req] GET /api/vehicles 200 6ms                 # recuperação automática
```

Mais de 195 mil requisições, nenhuma com erro

Cada cenário rodou por 10 segundos contra a API compilada, com tokens reais de cada perfil. O ambiente de teste roda API, banco e gerador de carga na mesma máquina; em produção a Vercel distribui a carga entre instâncias, como mostra a seção 14.

ROTA	CONEXÕES	REQ/S	MEDIANA	P90	P99	ERROS
`GET /health` (com o banco)	50	4.677	9 ms	15 ms	24 ms	0
`GET /api/vehicles`	10	874	10 ms	15 ms	21 ms	0
`GET /api/documents`	10	899	10 ms	14 ms	20 ms	0
`GET /api/expenses` (motorista)	10	718	13 ms	18 ms	25 ms	0
`GET /api/users/me`	10	648	14 ms	20 ms	26 ms	0
`GET /api/expenses` (administrador)	10	533	17 ms	25 ms	38 ms	0
`GET /api/dashboard/indicators`	10	408	23 ms	29 ms	38 ms	0
`POST /api/expenses` (gravação)	10	506	18 ms	24 ms	34 ms	0

Estresse do painel: de 10 a 200 conexões simultâneas



O painel é a rota mais pesada: soma despesas, manutenções e documentos do período. A vazão se mantém estável em torno de 400 painéis por segundo do começo ao fim, e mesmo com 200 conexões contínuas nenhuma requisição falhou.

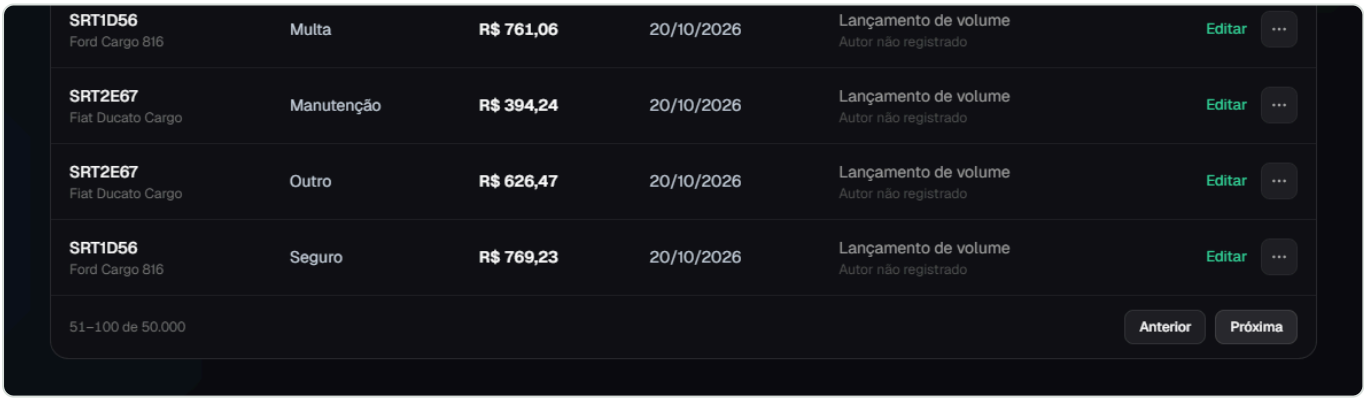
Uma empresa com 50 mil despesas

O cenário simula uma frota com anos de uso: 50 mil despesas numa única empresa, distribuídas por um ano. O objetivo é confirmar que o tempo de resposta não depende de quanto a empresa acumulou.

Como o sistema lida com volume

As listagens de despesas e manutenções são **paginadas**: a API entrega 50 registros por vez (até 200, se pedido) e informa o total no cabeçalho `X-Total-Count`. A página é escolhida por uma leitura estreita da tabela, só com os identificadores, e as colunas completas são montadas apenas para as linhas da página. Por isso a resposta tem tamanho fixo, cerca de 28 KB, com 500 ou com 50 mil lançamentos. O painel faz as somas no próprio banco e devolve só os totais.

CENÁRIO COM 50 MIL DESPESAS	RESULTADO
Lista de despesas, primeira página (50 de 50.000)	17 ms · 28 KB
Lista de despesas, última página	30 ms · 28 KB
Lista com filtro de período (últimos 30 dias)	11 ms
Painel no período padrão da tela (180 dias)	21 ms
Tela de despesas aberta no navegador	0,69 s
Painel aberto no navegador	0,75 s



Tela de despesas com 50 mil lançamentos, na segunda página ("51–100 de 50.000"): 50 lançamentos por vez, com Anterior e Próxima.

CARGA COM 50 MIL DESPESAS	CONEXÕES	REQ/S	MEDIANA	P99	ERROS
Lista, primeira página	10	413	23 ms	40 ms	0
Lista, primeira página	50	429	112 ms	157 ms	0
Lista, última página	10	161	58 ms	109 ms	0
Lista do motorista	10	729	13 ms	24 ms	0
Painel, período padrão (180 dias)	10	103	86 ms	138 ms	0

12 · RESILIÊNCIA

Se o banco cair, a API avisa e se recupera sozinha

Com a API no ar, o banco de dados foi pausado e depois retomado. O objetivo é ver como o sistema se comporta diante de uma dependência fora do ar.

MOMENTO	REQUISIÇÃO	RESPOSTA	TEMPO
Banco fora	`GET /health`	`503` · `"database": "unavailable"`	3,0 s
Banco fora	`GET /api/vehicles`	`503` · `Service unavailable`	10,0 s
Banco fora	`GET /api/vehicles` sem token	`401`: o processo segue atendendo	1 ms
Banco de volta	`GET /health`	`200` · `"database": "ok"`	4 ms
Banco de volta	`GET /api/vehicles`	`200`, sem reiniciar nada	7 ms

Resposta clara

O cliente recebe `503`, o código padrão para "serviço temporariamente indisponível", e não um erro genérico. Um monitor externo chamando o `/health` identifica a queda em 3 segundos.

Recuperação automática

Quando o banco volta, a primeira requisição já é atendida normalmente. Nenhum processo precisa ser reiniciado e nenhuma requisição fica presa.

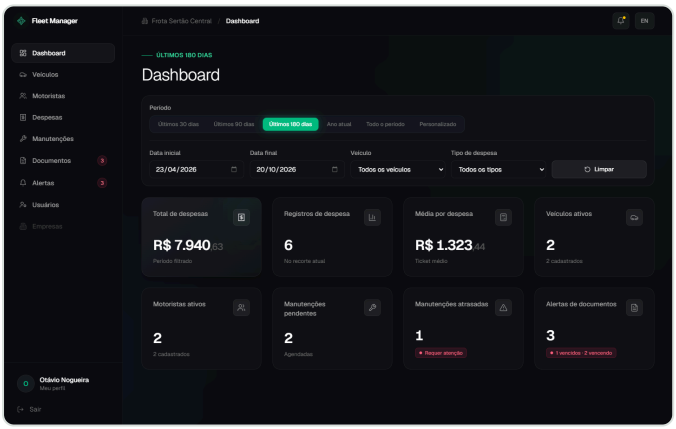
13 · TELAS NO NAVEGADOR

Cada perfil vê as próprias telas, sem erro

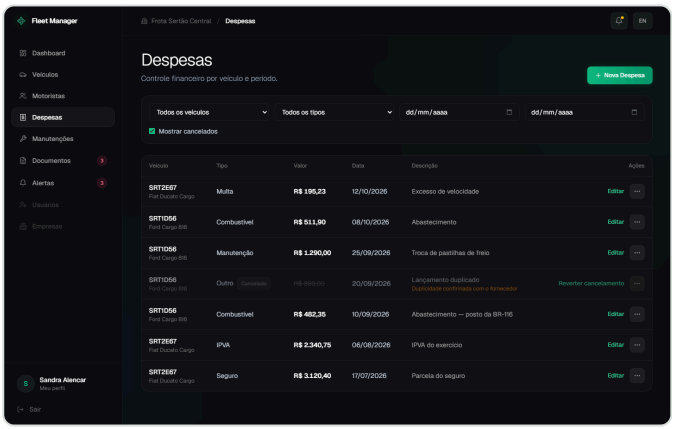
O frontend compilado foi aberto no Chrome automatizado, falando com a API e o banco. Cada perfil entrou pela tela de login e abriu as telas do próprio menu. O critério é nenhum erro de tela, de console ou de requisição.

PERFIL	TELAS ABERTAS	RESULTADO
Administrador	Painel, veículos, motoristas, despesas, manutenções, documentos, alertas, usuários, perfil	9 / 9
Gerente	Painel, veículos, motoristas, despesas, manutenções, documentos, alertas, usuários	8 / 8
Motorista	Painel, veículos, despesas, manutenções, documentos, perfil	6 / 6
Super administrador	Empresas, perfil	2 / 2
Pendente de aprovação	Acesso bloqueado até a aprovação (`403 PENDING_APPROVAL`)	Bloqueado

Cada tela ficou estável entre 0,61 e 0,73 s, incluindo a animação de entrada.



Painel do administrador, com os indicadores da frota.



Despesas do gerente, com um lançamento cancelado e o motivo.

14 · VERIFICAÇÃO EM PRODUÇÃO

O sistema publicado responde como o testado

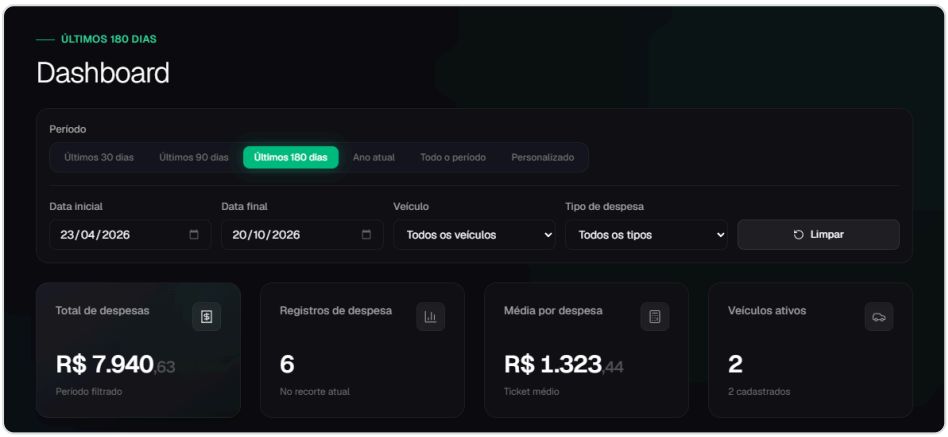
A produção foi verificada com contas de demonstração, só com leituras: rotas por perfil, permissões, telas e uma carga moderada, para não consumir a cota do plano. As sessões de teste foram encerradas ao final.

25 / 25
telas abertas sem erro, 4 perfis

11.840
requisições de carga, todas com status 200

~90 ms
mediana de resposta da API, dos quais ~70 ms são rede

16 / 16
publicações na Vercel concluídas



Painel em produção, 20/10/2026.

TESTE EM PRODUÇÃO	RESULTADO
8 rotas com administrador, gerente e motorista (24 chamadas)	Todas como esperado, de 0,23 a 0,52 s; o motorista recebe `403` na lista de usuários
Administrador lê veículo de outra empresa	`404`: o registro não aparece
Super administrador sem empresa escolhida	`403 COMPANY_NOT_SELECTED`; a lista de empresas responde `200`
Sem token · token malformado · token sem assinatura	`401` · `401` · `403` do firewall
Rotina diária de alertas	`ok` em 0,24 s

CARGA EM PRODUÇÃO	CONEXÕES	REQ/S	MEDIANA	P99	ERROS
`GET /api/vehicles`	10	105	90 ms	228 ms	0
`GET /api/expenses`	10	106	89 ms	227 ms	0
`GET /api/dashboard/indicators`	10	103	91 ms	268 ms	0
`GET /api/dashboard/indicators`	25	244	91 ms	396 ms	0

Ao passar de 10 para 25 conexões no painel, a vazão subiu de 103 para 244 req/s e a mediana não mudou. A Vercel abre mais instâncias conforme a demanda, em vez de colocar as requisições em fila.

Como os testes são escritos

Ataque de confusão de algoritmo (AUT-04). O teste forja um token HS256 com o segredo antigo. A API só aceita ES256 verificado pela chave pública e precisa recusar com 401.

```
it('AUT-04 token HS256 com o segredo legado → 401 (confusão de algoritmo)', async () => {
  const segredo = new TextEncoder().encode('super-secret-jwt-token-with-at-least-32-characters-long');
  const forjado = await new SignJWT({ email: PESSOAS.helena, role: 'authenticated' })
    .setProtectedHeader({ alg: 'HS256' })
    .setIssuer(`${SUPABASE_URL}/auth/v1`)
    .setAudience('authenticated')
    .setExpirationTime('1h')
    .sign(segredo);
  expect((await cliente(forjado).get('/users/me')).status).toBe(401);
});
```

Matriz de permissões (RBAC). Uma tabela gera um teste por linha: cada papel tenta o que não pode fazer e deve receber 403.

```
const negados: [Pessoa, string, string][] = [
  ['claudia', 'post',   '/vehicles'],                // motorista não cadastra veículo
  ['rogerio', 'patch',  `/users/${ID}/role`],        // gerente não muda papel
  ['helena',  'post',   '/companies'],               // admin não cria empresa
  // … 20 casos no total
];
it.each(negados)('RBAC %s %s %s → 403', async (pessoa, metodo, caminho) => {
  expect((await como(pessoa)[metodo](caminho)).status).toBe(403);
});
```

Paginação sem repetição (PAG-02). Percorre todas as páginas e confere que cada registro aparece exatamente uma vez e que a soma bate com o total do cabeçalho.

```
it('PAG-02 páginas seguidas não repetem registros e somam o total', async () => {
  const total = Number((await como('helena').get('/expenses?limit=1')).headers['x-total-count']);
  const vistos = new Set<string>();
  for (let offset = 0; offset < total; offset += 2) {
    const pagina = (await como('helena').get(`/expenses?limit=2&offset=${offset}`)).body;
    pagina.forEach((r) => vistos.add(r.id));
  }
  expect(vistos.size).toBe(total);
});
```

Verificação de saúde com prazo. A consulta ao banco corre contra um relógio de 3 s; a falha é tratada na própria promessa, para que uma resposta tardia nunca derrube o processo.

```
export async function bancoResponde(consultar = () => sql`select 1`, prazoMs = 3_000) {
  let relogio: NodeJS.Timeout | undefined;
  const prazo = new Promise<false>((resolve) => { relogio = setTimeout(() => resolve(false), prazoMs); });
  const consulta = consultar().then(() => true, () => false);
  try {
    return await Promise.race([consulta, prazo]);
  } finally {
    clearTimeout(relogio);
  }
}
```

O que um gestor de frota costuma perguntar

Outra empresa consegue ver os dados da minha frota?

Não. Toda consulta é filtrada pela empresa do usuário, lida do banco e não do que o navegador envia. Os testes de isolamento tentam ler, alterar e excluir dados de outra empresa, e recebem 404, inclusive em produção.

E se alguém tentar falsificar o acesso?

Os 13 testes de autenticação tentam os ataques conhecidos contra tokens: assinatura com outra chave, troca de algoritmo, token sem assinatura e chave pública usada como token. Todos são recusados. Em produção, o firewall da Vercel barra parte deles antes mesmo de chegarem à aplicação.

O motorista vê os gastos dos colegas?

Não. O motorista vê só os veículos vinculados a ele e os lançamentos que ele mesmo fez, e não vê a CNH de colegas.

Quantas pessoas podem usar ao mesmo tempo?

No teste, a API atendeu cerca de 400 painéis e de 500 a 900 consultas por segundo, sem erro, com até 200 conexões contínuas. Em produção, a plataforma abre mais servidores conforme a demanda: com 25 conexões simultâneas, a vazão mais que dobrou e o tempo de resposta não mudou.

E depois de anos de lançamentos?

As listagens trazem 50 registros por vez, com navegação entre páginas, e o painel faz as somas no banco. Com 50 mil despesas numa empresa, a lista responde em 17 ms e a tela abre em menos de 1 segundo.

Quanto tempo uma tela leva para abrir?

Entre 0,6 e 0,8 s no ambiente de teste e entre 0,9 e 2,1 s em produção, contando a animação de entrada. Cada consulta à API em produção leva cerca de 90 ms.

Meus documentos anexados ficam seguros?

Os arquivos ficam num armazenamento privado. A API só gera um link temporário, válido por 60 segundos, depois de conferir a mesma permissão do documento.

Dá para saber quem alterou um lançamento?

Sim. Cada alteração e cancelamento grava autor, data, campos alterados e motivo. O lançamento cancelado continua visível, riscado, com a justificativa.

E se o servidor de banco cair?

A API responde "serviço indisponível" em segundos, sem travar, e volta a funcionar sozinha assim que o banco retorna, sem reiniciar nada. A verificação de saúde acusa a queda em 3 segundos.

Como sabemos que uma atualização não quebrou nada?

Cada mudança passa pelo CI: tipagem, lint, build e os 373 testes, num banco criado do zero, em cerca de 2 minutos. A publicação na Vercel guarda todas as versões, e qualquer uma pode voltar ao ar com um clique.

ANEXO · COMO REPRODUZIR

Os mesmos testes, em qualquer máquina

```
# com o Docker aberto, na raiz do repositório
echo '[]' > supabase/signing_keys.json
npx supabase gen signing-key --algorithm ES256 --append
npx supabase start -x studio,imgproxy,vector,logflare,edge-runtime,realtime,mailpit,supavisor,postgres-meta
npm run db:migrate && npm run db:seed

npm run test:api        # 246 testes
npm run test:e2e        # 127 casos
npm run lint            # 0 avisos
npm audit               # 0 vulnerabilidades
```

Teste de carga. Cada cenário dispara requisições autenticadas por 10 s e registra vazão, percentis e erros.

```
const r = await autocannon({
  url: 'http://localhost:3000/api/dashboard/indicators',
  connections: 200, duration: 10,
  headers: { authorization: `Bearer ${token('admin')}` },
});
// → 402 req/s · mediana 473 ms · p99 601 ms · 0 erros · 0 status fora de 2xx
```

FERRAMENTA	VERSÃO
Node.js	24.13 (local) · 22 (CI)
Vitest	4.1.12
Supabase CLI · PostgreSQL	2.121.0 · 17.11
autocannon · Playwright	7.15 · 1.48
Docker	29.4